UC 00605

Relatório do projeto final

Hudson Peres

Tássia Nascimento

2026

1. Introdução.....3
2. Objetivos do Projeto.....3
3. Arquitetura do Sistema.....4
4. Descrição dos Componentes.....4
5. API REST.....5
6. Base de Dados.....5
7. Sistema de Cache (Redis).....6
8. Polly (Cache e Resiliência).....6
9. Imposter (Mock Externo).....6
10. Autenticação e Segurança.....7
11. Frontend / Aplicação Mobile.....7
12. Fluxo de Funcionamento e Implementação.....8
13. Deploy.....8
14. Resultados e Conclusão.....9

1. Introdução

O projeto consistiu no desenvolvimento de uma API REST para gestão de utilizadores e produtos, como trabalho final da UC00605. A aplicação foi projetada para ser consumida por um website e/ou aplicação móvel, com foco na segurança, desempenho e resiliência. O ecossistema tecnológico empregado compreende:

- Back-end: ASP.NET Core 8 (C#).
- Base de Dados: PostgreSQL com Entity Framework Core.
- Cache: Redis para cache distribuído.
- Resiliência: Polly para cache local, retries e circuit breaker.
- Simulação: Mountebank como imposter (mock) para inventário e pagamentos.
- Autenticação: JWT.
- Frontend: HTML, CSS e JavaScript puro.
- Orquestração: Docker Compose.

2. Objetivos do Projeto

O foco central do projeto é o desenvolvimento de uma infraestrutura de backend robusta, capaz de suportar múltiplas requisições simultâneas. Esta base deve garantir, primordialmente, a integridade dos dados processados e a sua persistência segura a longo prazo no banco de dados.

Para isto, foram definidos os seguintes objetivos especificos:

- Segurança: Implementação de autenticação JWT, com registo e login de utilizadores (passwords com hash BCrypt).
- Performance: Otimização do tempo de resposta através de cache híbrido (memória local + Redis).
- Disponibilidade: Garantia de operação mesmo diante de falhas do mock externo, usando retries e circuit breaker (Polly).
- Simulação de serviços externos: Criação de um imposter com Mountebank para os endpoints de inventário e pagamentos.
- Frontend demonstrativo: Desenvolvimento de uma interface simples (HTML/CSS/JS) que consome todos os endpoints da API.
- Deploy simplificado: Orquestração com Docker Compose, permitindo executar toda a stack com um único comando.

3. Arquitetura do Sistema

A arquitetura adotada para o desenvolvimento da API foi a arquitetura em Camadas, esta escolha visa a separação de responsabilidades, facilitando a manutenção e os testes.

A organização do sistema está dividida em:

- Apresentação (Controllers): Gerem os pedidos HTTP, validam entradas e retornam respostas JSON.
- Serviços (Services): Contêm a lógica de cache híbrido (HybridCacheService, RedisCacheService) e as políticas de resiliência.
- Dados (Data): Contexto do Entity Framework (AppDbContext) e migrações para acesso ao PostgreSQL.
- Todo o ambiente foi orquestrado com Docker Compose, isolando a API, PostgreSQL, Redis e Mountebank em contentores.

4. Descrição dos Componentes

O sistema é composto por módulos especializados que garantem o fluxo correto das informações e a segurança da operação:

- Auth Controller: Responsável pelo registo e login de utilizadores, emissão e validação de tokens JWT.
- Products Controller: Expõe operações CRUD para produtos, utilizando cache híbrido (local + Redis) nos endpoints de leitura.
- Users Controller: CRUD completo para utilizadores, apenas acessível com token JWT.
- HybridCacheService: Coordena o cache de dois níveis: primeiro memória local, depois Redis e a invalidação.
- RedisCacheService: Abstração para operações com Redis.
- ResiliencePolicies: Define as políticas de retry, 3 tentativas com backoff exponencial, e circuit breaker, abre após 5 falhas.
- InventoryController / PaymentsController: Consomem o mock Mountebank através de um HttpClient resiliente.
- Mountebank: Imposter que simula os endpoints: /inventory/{sku} e /payments.

5. API REST

A API disponibiliza os seguintes grupos de endpoints:

- Auth: POST /api/Auth/register para registo, POST /api/Auth/login que devolve token JWT.
- Users: GET, GET/{id}, POST, PUT/{id}, DELETE/{id}.
- Products: GET, GET/{id}, POST, PUT/{id}, DELETE/{id}.
- Inventory: GET /api/Inventory/{sku} (consumo do mock).
- Payments: POST /api/Payments (consumo do mock).

Todos os endpoints, exceto registo e login, exigem autenticação por token JWT, enviado no cabeçalho Authorization: Bearer <token>. O formato de intercâmbio é JSON.

6. Base de Dados

Foi utilizado o PostgreSQL como sistema de base de dados relacional. O acesso é feito através do Entity Framework Core. As principais tabelas são:

Os principais aspectos da modelação de dados incluem:

- Users (Id, Username, PasswordHash, Email)
- Products (Id, Name, Price, Stock).

As migrações do EF Core gerenciam o esquema, e o método dbContext.Database.Migrate() é executado automaticamente no arranque da aplicação para garantir que as tabelas estejam criadas

7. Sistema de Cache - Redis

O Redis é utilizado como cache distribuído. As chaves típicas são products_all para lista de produtos e product_{id} para produto individual. O tempo de expiração padrão é de 5 minutos. A interação é feita através do RedisCacheService, que serializa objetos em JSON. O cache reduz drasticamente o número de acessos à base de dados e acelera as respostas da API.

8. Polly - cache e resiliência

O Polly é usado para dois propósitos:

- Cache local em memória: Primeiro nível de cache, com expiração de 30 segundos, implementado via IMemoryCache e integrado no HybridCacheService.
- Resiliência nas chamadas ao Mountebank:
  - Retry: até 3 tentativas com backoff exponencial.
  - Circuit Breaker: após 5 falhas consecutivas, o circuito abre por 30 segundos, bloqueando novas chamadas ao mock e retornando erro imediato.

Estas políticas garantem que a API não fique inoperante caso o imposter esteja temporariamente indisponível.

9. Imposter - Mock externo

foi utilizado o Mountebank para simular dois serviços externos:

- Inventário: GET /inventory/{sku} responde com {"sku":"123","quantity":42} (fixo).
- Pagamentos: POST /payments responde com {"status":"paid","transactionId":"mock-txn-123"}.

A configuração está no ficheiro imposters/inventory.json. O Mountebank corre num contentor Docker, e a API chama-o através de um HttpClient resiliente (com as políticas Polly). Os endpoints expostos são /api/Inventory/{sku} e /api/Payments.

10. Autenticação e Segurança

A segurança é baseada em JWT, seguindo o fluxo:

1. O utilizador regista-se (/api/Auth/register) e a password é guardada com hash BCrypt.
2. O utilizador faz login (/api/Auth/login) e a API verifica o hash e devolve um token JWT assinado (expira em 1 hora).
3. Nos pedidos seguintes, o token deve ser enviado no cabeçalho Authorization.
4. O middleware de autenticação do ASP.NET Core valida o token (assinatura, emissor, audiência, expiração) antes de executar os controladores protegidos com [Authorize].

Não foi implementado controle baseado em roles. A segurança se concentra na autenticação de utilizadores.

11. Frontend / Aplicação Mobile

Foi desenvolvido um frontend demonstrativo em HTML5, CSS3 e JavaScript puro (três ficheiros: index.html, style.css, main.js). As funcionalidades incluem:

- Tela de login (obtenção e armazenamento do token JWT no localStorage).
- Listagem de produtos com layout de cartões responsivo.
- Criação, edição e eliminação de produtos (com modais).
- Consulta do inventário simulado (chamada ao endpoint /api/Inventory/123).

A comunicação com a API é feita via fetch, enviando o token em cada pedido. O frontend é estático e pode ser aberto diretamente no navegador (desde que a API tenha CORS configurado).

12. Fluxo de Funcionamento e Implementação

O percurso de uma requisição de leitura:

1. O cliente (frontend ou Postman) envia GET /api/Products com token JWT.
2. A API valida o token (middleware de autenticação).
3. O ProductsController chama o HybridCacheService.GetOrSetAsync.
4. Primeiro verifica o cache local (memória).
5. Se não existir, verifica o Redis.
6. Se ainda não existir, consulta a base de dados PostgreSQL.
7. O resultado é armazenado no Redis e no cache local, e depois devolvido ao cliente.

Para chamadas ao mock (inventário/pagamentos), o HttpClient resiliente aplica as políticas de retry e circuit breaker antes de aceder ao Mountebank.

A implementação seguiu boas práticas como separação de responsabilidades, injeção de dependências, configuração externa e uso de variáveis de ambiente no Docker.

13. Deploy

O deploy é feito inteiramente com Docker. O ficheiro docker-compose.yml define quatro serviços:

- postgres (PostgreSQL 15)
- redis (Redis Alpine)
- mountebank (com a configuração do imposter)
- api (imagem construída a partir do Dockerfile da API)

Comando único: docker-compose up --build. Iso levanta todos os contentores, interliga-os na mesma rede e expõe as portas:

- API: http://localhost:5000/swagger
- Mountebank: http://localhost:8000
- PostgreSQL: 5432, Redis: 6379

A base de dados persiste através de um volume Docker. Este modelo garante consistência entre ambientes de desenvolvimento e produção.

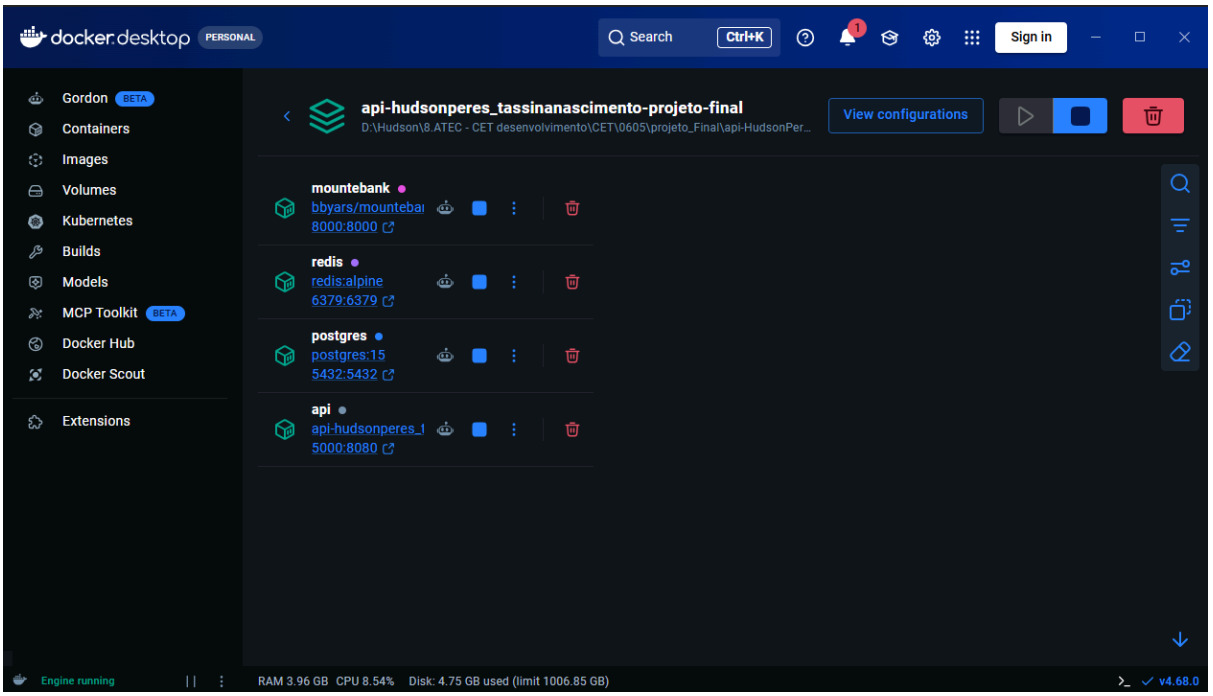
14. Resultados e Conclusão

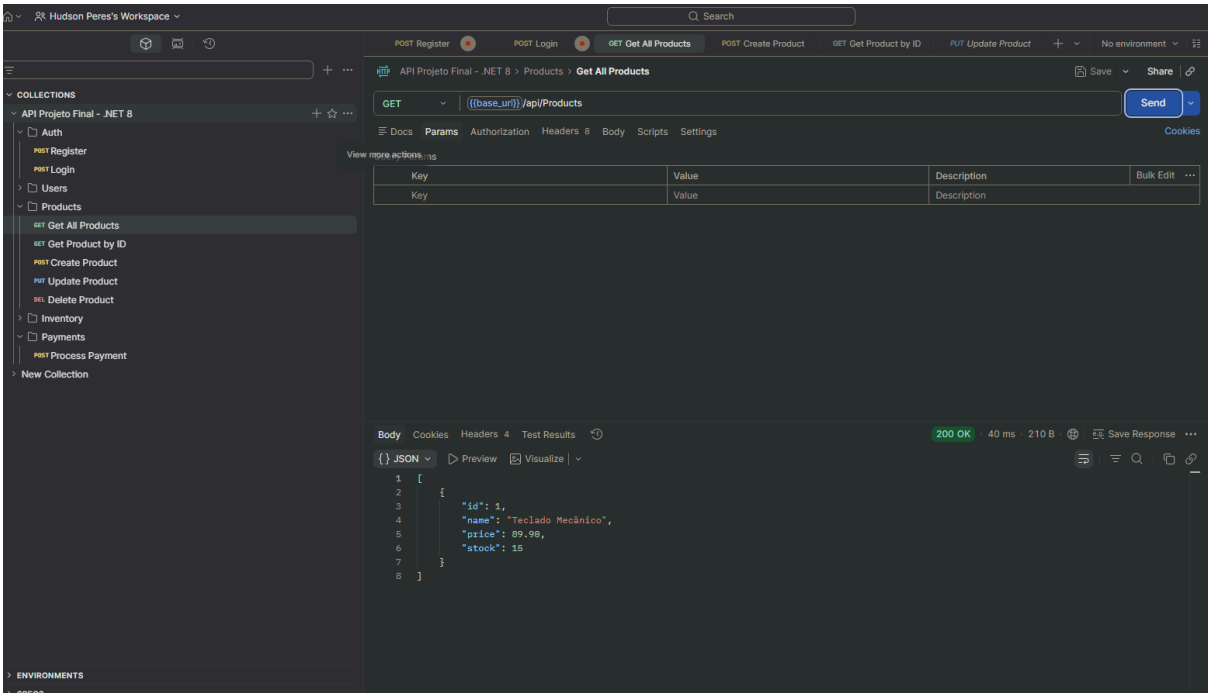
O projeto entregou uma API completamente funcional, validada por testes manuais no Swagger e via Postman, e também por um frontend real. Os resultados práticos incluem:

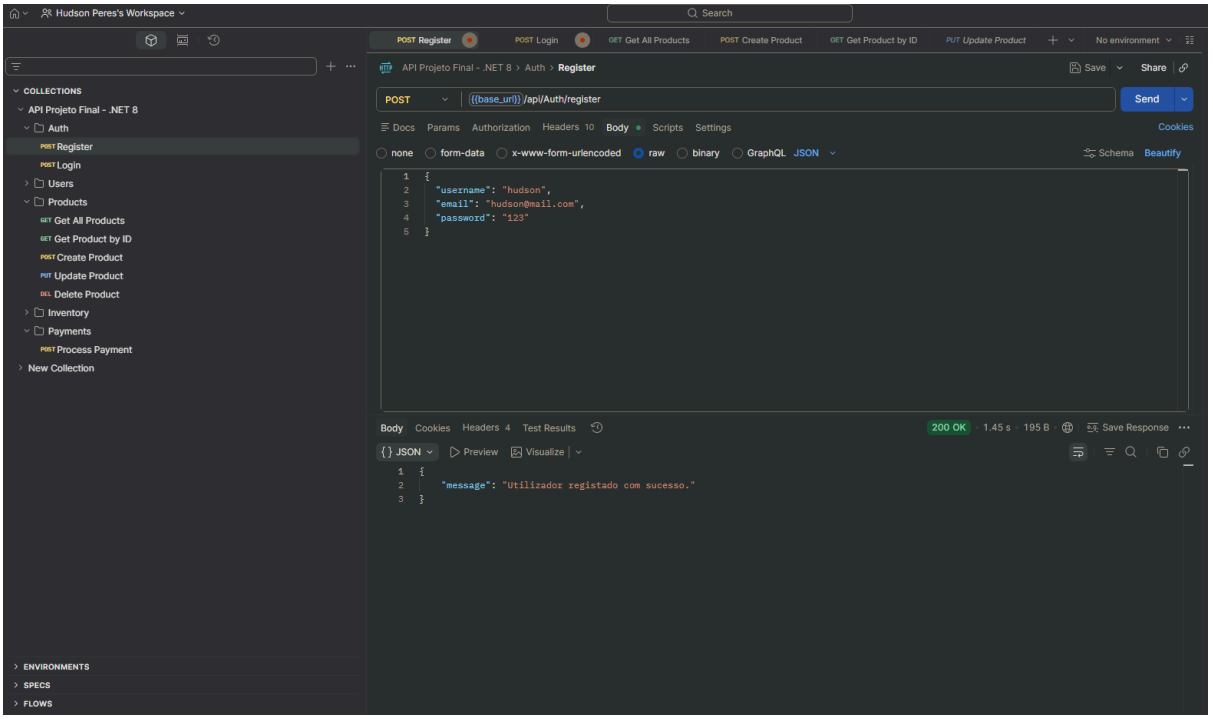
- Segurança: Autenticação JWT funcional, com registo e login seguros (BCrypt).
- Cache híbrido: Demonstrou ganhos significativos de velocidade (a primeira consulta é lenta da base de dados; as seguintes são rápidas).
- Resiliência: O sistema não trava quando o Mountebank é desligado. Executa retries e ativa o circuit breaker.
- Imposter: A simulação de inventário e pagamentos permite testar a integração sem dependências externas.
- Frontend: Interface simples mas completa, provando que a API é facilmente consumível.
- Docker Compose: A orquestração simplifica o arranque do projeto por qualquer avaliador.

O projeto entrega uma infraestrutura de backend preparada para integração com interfaces modernas. A utilização de padrões como Injeção de Dependência, código devidamente estruturado e entendível e a contentorização garante que o produto final seja escalável e de fácil manutenção.

14. Anexos e imagens




Hudson Peres's Workspace
Search
POST Register
POST Login
GET Get All Products
POST Create Product
GET Get Product by ID
PUT Update Product
No environment
COLLECTIONS
API Projeto Final - .NET 8
Auth
POST Register
POST Login
Users
Products
GET Get All Products
GET Get Product by ID
POST Create Product
PUT Update Product
DEL Delete Product
Inventory
Payments
POST Process Payment
New Collection
ENVIRONMENTS
API Projeto Final - .NET 8 > Products > Get All Products
Save
Share
GET
{{base_url}}/api/Products
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
Key
Value
Description
Bulk Edit
Body
Cookies
Headers 4
Test Results
200 OK
40 ms
210 B
Save Response
JSON
Preview
Visualize
[
{
"id": 1,
"name": "Teclado Mecânico",
"price": 89.90,
"stock": 15
}
]


Hudson Peres's Workspace
Search
POST Register
POST Login
GET Get All Products
POST Create Product
GET Get Product by ID
PUT Update Product
No environment
COLLECTIONS
API Projeto Final - .NET 8
Auth
POST Register
POST Login
Users
Products
GET Get All Products
GET Get Product by ID
POST Create Product
PUT Update Product
DEL Delete Product
Inventory
Payments
POST Process Payment
New Collection
ENVIRONMENTS
SPECS
FLOWS
API Projeto Final - .NET 8 > Auth > Register
Save
Share
POST
{{base_url}}/api/Auth/register
Send
Docs
Params
Authorization
Headers 10
Body
Scripts
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Beautify
{
"username": "hudson",
"email": "hudson@mail.com",
"password": "123"
}
Body
Cookies
Headers 4
Test Results
200 OK
1.45 s
195 B
Save Response
JSON
Preview
Visualize
{
"message": "Utilizador registado com sucesso."
}

Swagger

Select a definition: api-HudsonPeres_TassinaNascimento-projeto-final v1

Minha API

v1 OAS3

http://localhost:5144/swagger/v1/swagger.json

Authorize

Auth

POST /api/Auth/register

Parameters — Cancel — Reset

No parameters

Request body — application/json

```
{
  "username": "admin99",
  "email": "admin@mail.com",
  "password": "12345A"
}
```

Execute — Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5144/api/Auth/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "username": "admin99",
  "email": "admin@mail.com",
  "password": "12345A"
}'
```

Request URL

http://localhost:5144/api/Auth/register

Server response

Code	Details
200	Response body: `{ "message": "Utilizador registado com sucesso." }` Download

Response headers

```
access-control-allow-origin: *
content-type: application/json; charset=utf-8
date: Thu,09 Apr 2026 19:59:38 GMT
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
200	Success	No links

POST /api/Auth/login

Parameters — Cancel — Reset

No parameters

Request body — application/json

```
{
  "username": "admin99",
  "password": "12345A"
}
```

Execute — Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5144/api/Auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "username": "admin99",
  "password": "12345A"
}'
```

Request URL

http://localhost:5144/api/Auth/login

Server response

Code	Details
200	Response body: `{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCIsIkpXVCJ9.eyJuYWlkIjoiMjIiLCJlcmFpbCI6IjAiLCJyb2xlIjoiMSIsIkFkbWluaXN0cmFkb3JlIjoiMSIsIkNsaWVudCIsIkNsaWVudCIsIkNsaWVudC

Swagger

Select a definition api-HudsonPeres_TassinaNascimento-projeto-final v1

Minha API v1 OAS3

http://localhost:5144/swagger/v1/swagger.json

Authorize

Auth

POST /api/Auth/register

Parameters

Cancel Reset

No parameters

Request body

application/json

```
{
  "username": "admin99",
  "email": "admin99@mail.com",
  "password": "123456"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5144/api/Auth/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "username": "admin99",
  "email": "admin99@mail.com",
  "password": "123456"
}'
```

Request URL

http://localhost:5144/api/Auth/register

Server response

Code Details

Failed to fetch.
Possible Reasons:

- CORS
- Network Failure
- URL scheme must be "http" or "https" for CORS request.

Responses

Code	Description	Links
200	Success	No links

POST /api/Auth/login

Parameters

Cancel Reset

No parameters

Request body

application/json

```
{
  "username": "admin99",
  "password": "123456"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5144/api/Auth/login' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
  "username": "admin99",
  "password": "123456"
}'
```

Request URL

http://localhost:5144/api/Auth/login

Server response

Code Details

Failed to fetch.
Possible Reasons:

- CORS
- Network Failure
- URL scheme must be "http" or "https" for CORS request.

Responses

Code	Description	Links
200	Success	No links

Inventory

GET /api/Inventory/{sku}

Payments

POST /api/Payments

Products

GET /api/Products

POST /api/Products

GET /api/Products/{id}

PUT /api/Products/{id}

DELETE /api/Products/{id}

Teste

GET /api/Teste

Users

GET /api/Users

POST /api/Users

GET /api/Users/{id}

PUT /api/Users/{id}

DELETE /api/Users/{id}

Schemas

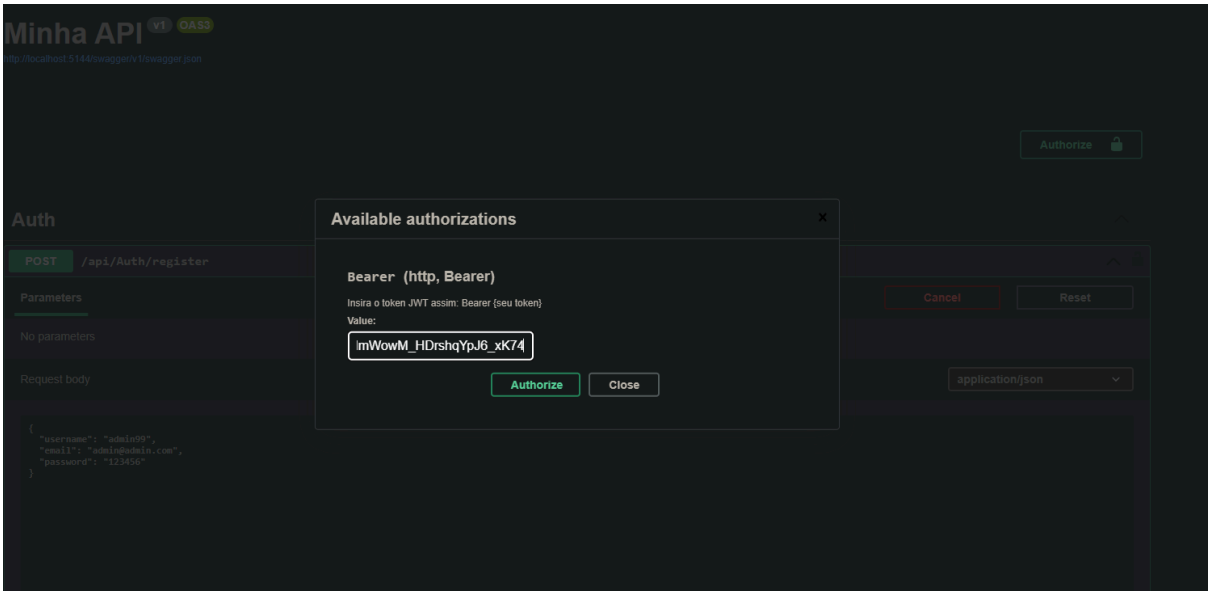
LoginRequest >

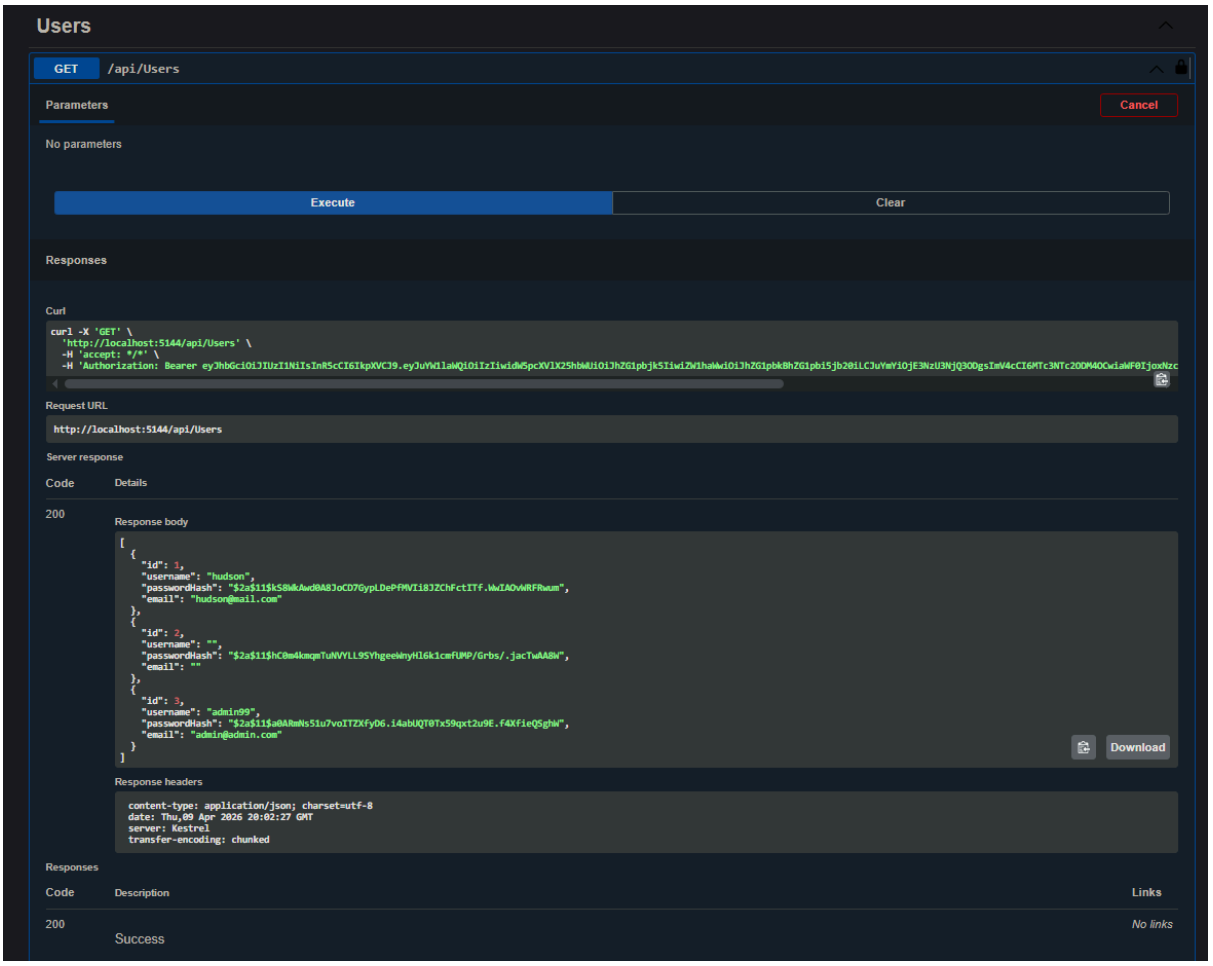
PaymentRequest >

Product >

RegisterRequest >

User >


Minha API
V1
OAS3
http://localhost:5144/swagger/v1/swagger.json
Authorize
Auth
POST
/api/Auth/register
Parameters
Cancel
Reset
No parameters
Request body
application/json
{
"username": "admin99",
"email": "admin@admin.com",
"password": "123456"
}
Available authorizations
Bearer (http, Bearer)
Insira o token JWT assim: Bearer {seu token}
Value:
mWowM_HDrshqYpJ6_xK74
Authorize
Close


Users
GET
/api/Users
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:5144/api/Users' \
-H 'accept: */*' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYW1laWQiOiIzIiwidW5pcXVlX25hbWUiOiJhZG1pbjk5IiwiZW1haWwiOiJhZG1pbk8hZG1pbi5jb20iLCJuYmYiOjE3NzU3NjQ3ODgsImV4cCI6MTc3NTc2ODM4OCwiaWF0IjoxNzc
Request URL
http://localhost:5144/api/Users
Server response
Code
Details
200
Response body
[
{
"id": 1,
"username": "hudson",
"passwordHash": "$2a$11$kS8wkAwd0A8JoCD7GypLDePfMVTi8JZChFctITf.WwIAOvWRFRwum",
"email": "hudson@mail.com"
},
{
"id": 2,
"username": "",
"passwordHash": "$2a$11$hC0m4kmqmTuNVYLL9SYhgeeWnyHl6k1cmfUMP/Grbs/.jacTwAA8W",
"email": ""
},
{
"id": 3,
"username": "admin99",
"passwordHash": "$2a$11$a0ARmNs51u7voITZXfyO6.i4abUQT0Tx59qxt2u9E.f4XfieQSghW",
"email": "admin@admin.com"
}
]
Download
Response headers
content-type: application/json; charset=utf-8
date: Thu,09 Apr 2026 20:02:27 GMT
server: Kestrel
transfer-encoding: chunked
Responses
Code
Description
Links
200
Success
No links

DELETE /api/Users/{id}

Parameters

Cancel

Name	Description
id * required integer($int32) (path)	2

Execute | Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:5144/api/Users/2' \
  -H 'accept: */*' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJuYWlaWQiOiIzIiwidW5pcXVlX25hbWUiOiJhZG1pbjk5IiwiZW1haWxiOiJhZG1pbkBhZG1pbi5jb20iLCJuYmYiOjE3NzU3NjQ3ODgsImV4cCI6MTc3NTc2ODM4OCwiaWF0IjoxNzc
```

Request URL

```
http://localhost:5144/api/Users/2
```

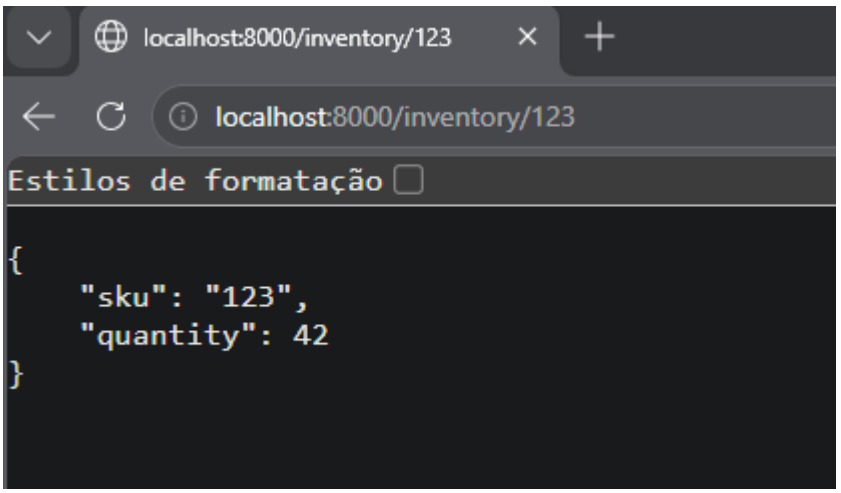
Server response

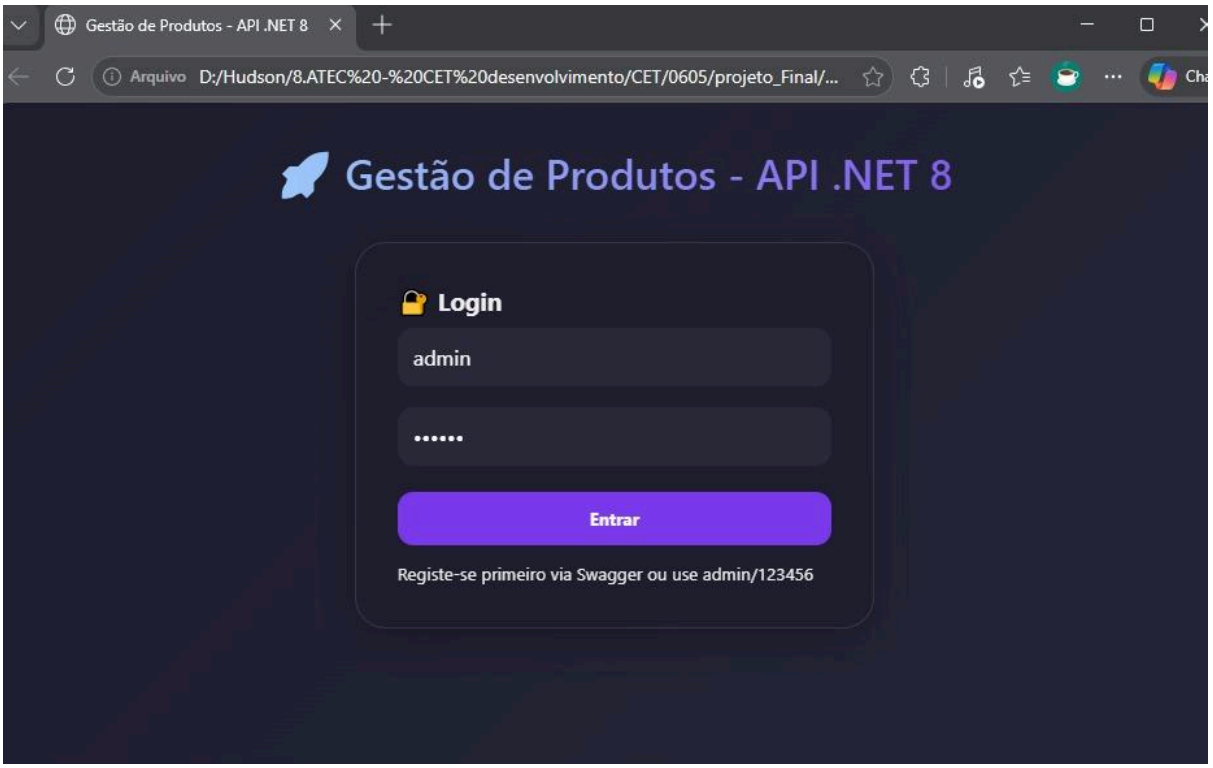
Code	Details
204 Undocumented	Response headers

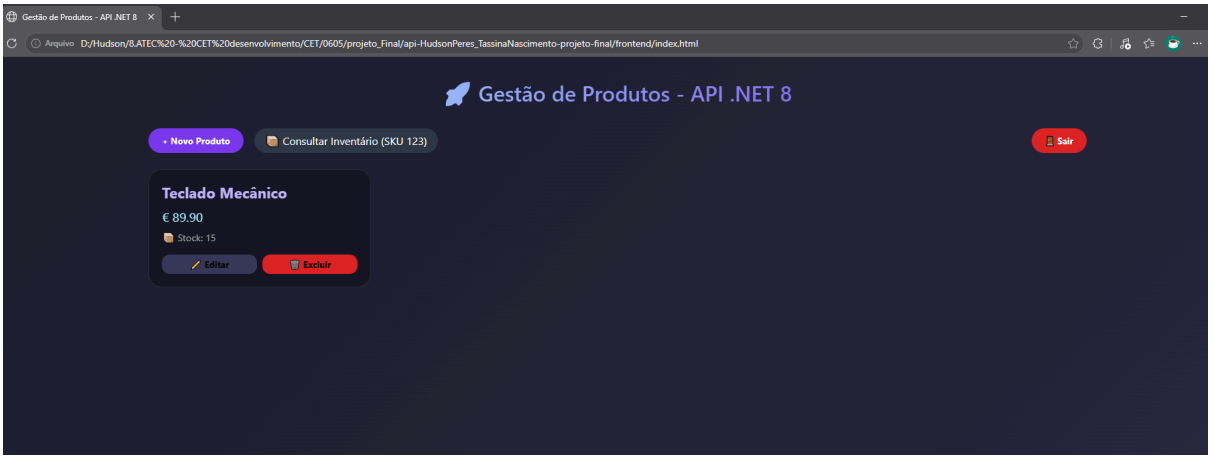
```
access-control-allow-origin: *
date: Thu,09 Apr 2026 20:03:38 GMT
server: Kestrel
```

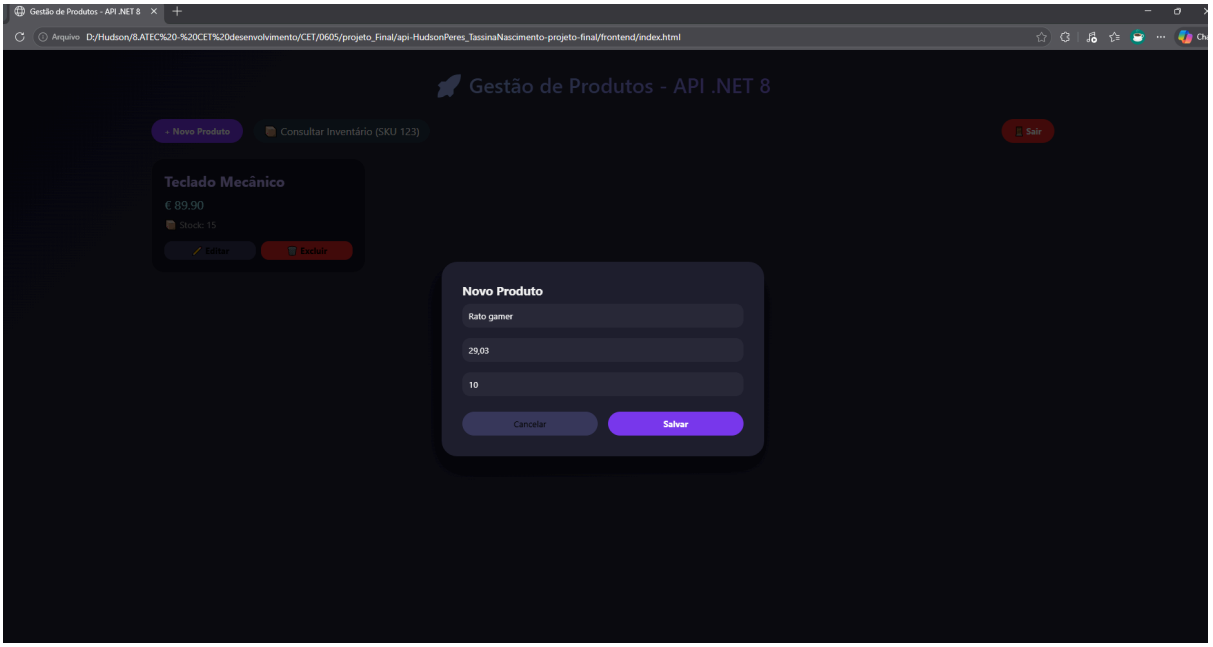
Responses

Code	Description	Links
200	Success	No links

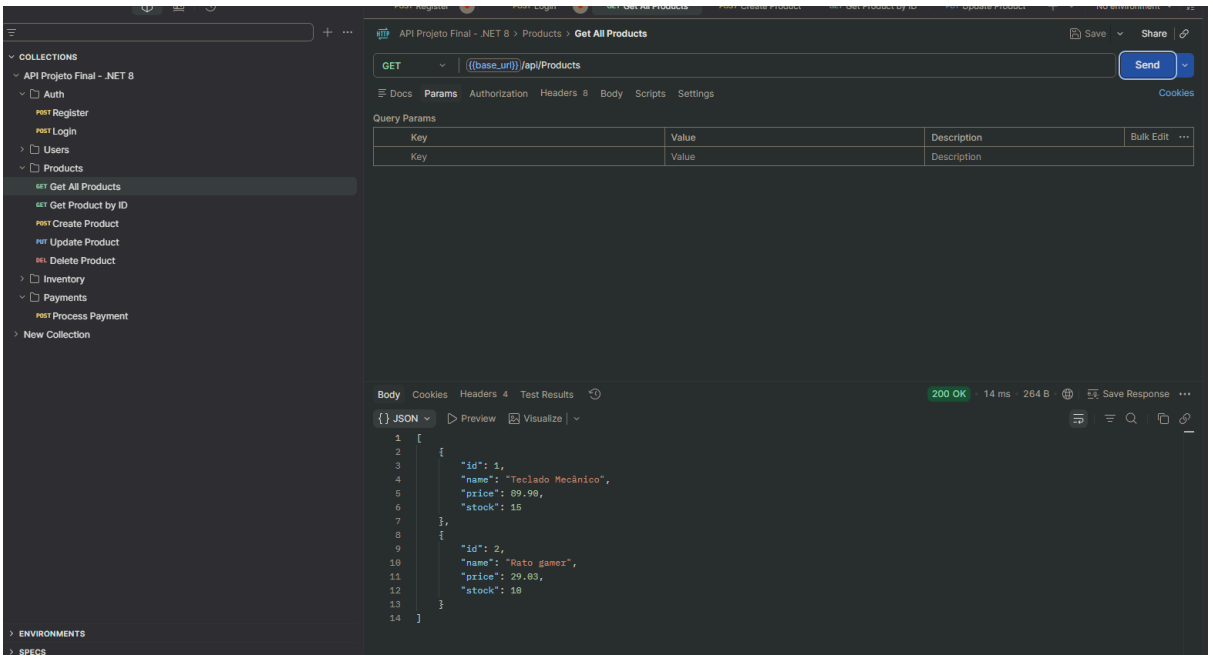

localhost:8000/inventory/123
localhost:8000/inventory/123
Estilos de formatação
{
"sku": "123",
"quantity": 42
}

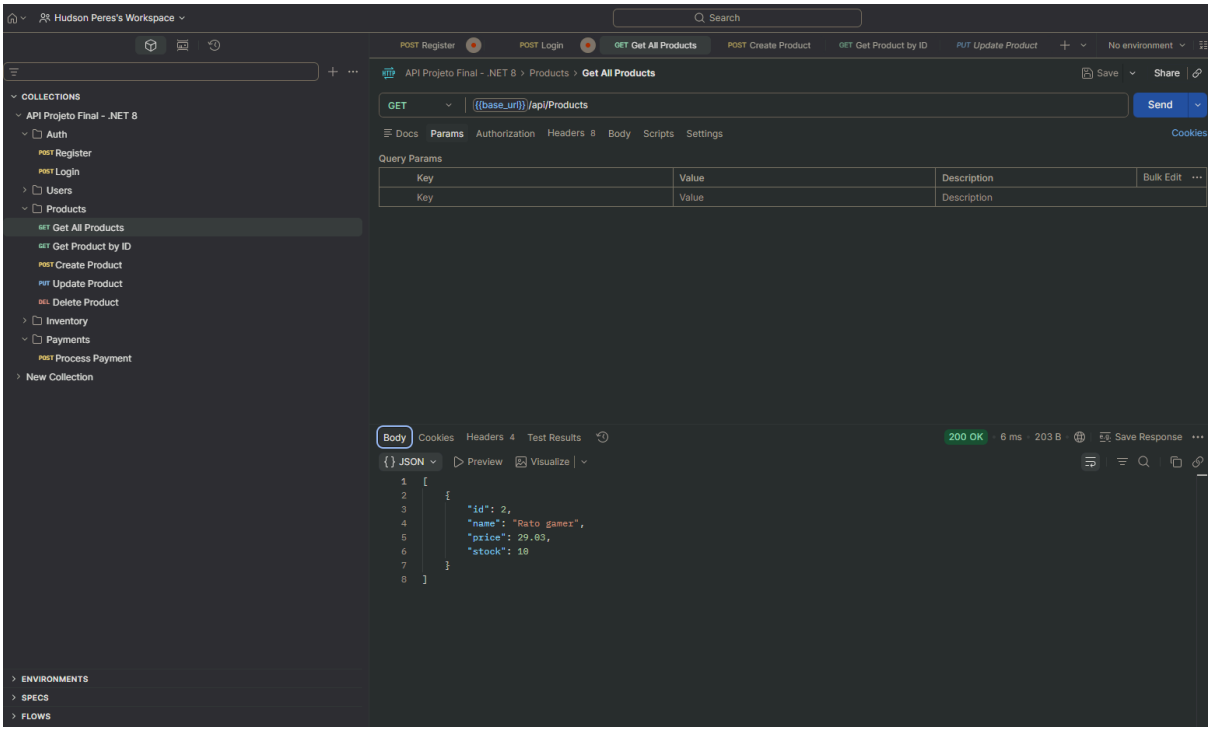

Gestão de Produtos - API .NET 8
Arquivo D:/Hudson/8.ATEC%20-%20CET%20desenvolvimento/CET/0605/projeto_Final/...
Gestão de Produtos - API .NET 8
Login
admin
••••••
Entrar
Registe-se primeiro via Swagger ou use admin/123456

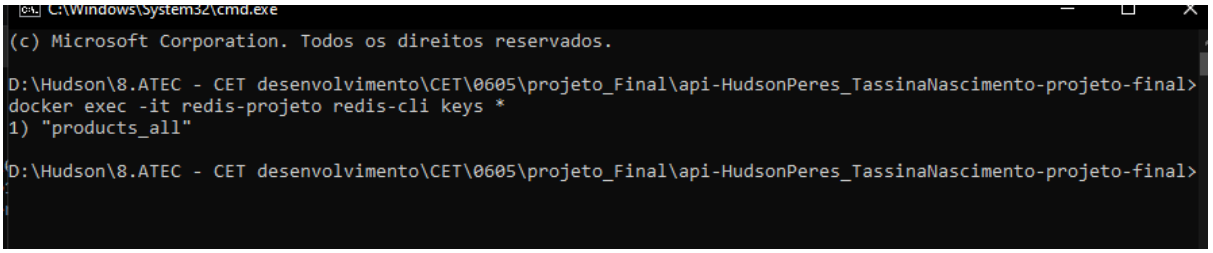

Gestão de Produtos - API .NET 8
Arquivo D:/Hudson/8.ATEC%20-%20CET%20desenvolvimento/CET/0605/projeto_Final/api-HudsonPeres_TassinaNascimento-projeto-final/frontend/index.html
Gestão de Produtos - API .NET 8
+ Novo Produto
Consultar Inventário (SKU 123)
Sair
Teclado Mecânico
€ 89.90
Stock: 15
Editar
Excluir


Gestão de Produtos - API .NET 8
Arquivo D:/Hudson/8.ATEC%20-%20CET%20desenvolvimento/CET/0605/projeto_Final/api-HudsonPeres_TassinaNascimento-projeto-final/frontend/index.html
Gestão de Produtos - API .NET 8
+ Novo Produto
Consultar Inventário (SKU 123)
Sair
Teclado Mecânico
€ 89.90
Stock: 15
Editar
Excluir
Novo Produto
Rato gamer
29,03
10
Cancelar
Salvar

Gestão de Produtos - API .NET 8
Arquivo D:/Hudson/8.ATEC%20-%20CET%20desenvolvimento/CET/0605/projeto_Final/api-HudsonPeres_TassinaNascimento-projeto-final/frontend/index.html
Gestão de Produtos - API .NET 8
+ Novo Produto
Consultar Inventário (SKU 123)
Sair
Teclado Mecânico
€ 89.90
Stock: 15
Editar
Excluir
Rato gamer
€ 29.03
Stock: 10
Editar
Excluir


COLLECTIONS
API Projeto Final - .NET 8
Auth
POST Register
POST Login
Users
Products
GET Get All Products
GET Get Product by ID
POST Create Product
PUT Update Product
DEL Delete Product
Inventory
Payments
POST Process Payment
New Collection
ENVIRONMENTS
SPECS
API Projeto Final - .NET 8 > Products > Get All Products
Save
Share
GET
{{base_url}} /api/Products
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Key
Value
Description
Body
Cookies
Headers 4
Test Results
200 OK
14 ms
264 B
Save Response
JSON
Preview
Visualize
[
{
"id": 1,
"name": "Teclado Mecânico",
"price": 89.90,
"stock": 15
},
{
"id": 2,
"name": "Rato gamer",
"price": 29.03,
"stock": 10
}
]


Hudson Peres's Workspace
Search
POST Register
POST Login
GET Get All Products
POST Create Product
GET Get Product by ID
PUT Update Product
No environment
COLLECTIONS
API Projeto Final - .NET 8
Auth
POST Register
POST Login
Users
Products
GET Get All Products
GET Get Product by ID
POST Create Product
PUT Update Product
DEL Delete Product
Inventory
Payments
POST Process Payment
New Collection
ENVIRONMENTS
SPECS
FLOWS
API Projeto Final - .NET 8 > Products > Get All Products
Save
Share
GET
{{base_url}}/api/Products
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
Query Params
Key
Value
Description
Bulk Edit
Body
Cookies
Headers 4
Test Results
200 OK
6 ms
203 B
Save Response
JSON
Preview
Visualize
[
{
"id": 2,
"name": "Rato gamer",
"price": 29.03,
"stock": 10
}
]


C:\Windows\System32\cmd.exe
(c) Microsoft Corporation. Todos os direitos reservados.
D:\Hudson\8.ATEC - CET desenvolvimento\CET\0605\projeto_Final\api-HudsonPeres_TassinaNascimento-projeto-final>
docker exec -it redis-projeto redis-cli keys *
1) "products_all"
D:\Hudson\8.ATEC - CET desenvolvimento\CET\0605\projeto_Final\api-HudsonPeres_TassinaNascimento-projeto-final>